

Received 30 June 2022, accepted 18 July 2022, date of publication 21 July 2022, date of current version 1 August 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3192879

## APPLIED RESEARCH

# On-Die Dynamic Remapping Cache: Strong and Independent Protection Against Intermittent Faults

SANGJAE PARK<sup>1</sup> AND JUNGRAE KIM<sup>2</sup>, (Senior Member, IEEE)

<sup>1</sup>Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon-si 16419, South Korea

<sup>2</sup>Department of Semiconductor Systems Engineering, Sungkyunkwan University, Suwon-si 16419, South Korea

Corresponding author: Jung-rae Kim (dale40@skku.edu)

This work was partly supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by Ministry of Science ICT (MSIT, Korea) (No. 2021-0-00863, Intelligent in-memory error-correction device for high-reliability memory, 50% / No.2019-0-00421, Artificial Intelligence Graduate School Program (Sungkyunkwan University), 30%) and National R&D Program through the National Research Foundation of Korea (NRF) funded by Ministry of Science ICT (MSIT, Korea) (No. 020R1C1C1011419, Scaling Out Distributed Artificial Intelligence with Network Optimization, 20%). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

**ABSTRACT** As process scaling continues, DRAM is getting more vulnerable to errors. System companies and DRAM vendors have introduced ECC to protect DRAM against growing errors. ECC, however, should be combined with a repair mechanism to prevent non-transient faults from repeatedly producing errors and to prevent overlapping faults from accumulating into more severe errors. We propose a novel approach to repair memory and improve reliability with minimal overheads. On-die Dynamic Remapping Cache (DRC) minimizes the repair overheads by focusing on active faults. Most intermittent faults (e.g., Variable Retention Time) generate errors occasionally, and the number of active faults at any one time is significantly lower than the total. DRC tracks the activity and severity of faults at run-time and uses a small cache inside a DRAM to remap active faults. This efficiency enables aggressive remapping of bit faults, which eliminates fault accumulations and improves reliability. Our evaluation shows DRC can provide much stronger protection than the state-of-the-art protection schemes with no performance degradation and a negligible chip area overhead.

**INDEX TERMS** Computer architecture, memory architecture, DRAM, reliability.

## I. INTRODUCTION

Dram reliability is a serious and growing concern. In modern data centers, DRAM errors are the second dominant cause of hardware failures [1]–[4]. The importance of DRAM reliability is further growing as mission-critical systems (e.g., autonomous driving) require higher reliability and systems deploy more DRAM chips. Despite the demand, individual DRAM chips are becoming more vulnerable to faults and errors [5]–[8]. Process scaling has shrunk DRAM cells into nm-scale feature size and fF-scale cell capacitance. At this atomic scale, a subtle difference in the operating conditions can trigger an error. As a result, intermittent faults (i.e., faults

that manifest errors occasionally) are growing in numbers, and they generate frequent bit-level errors. Combined with other faults [1]–[4], these intermittent faults will challenge DRAM reliability in future systems.

For high-reliability applications, system companies rely on *Rank-level ECC* (R-ECC) to improve memory reliability. R-ECC utilizes extra chips on each ECC Dual In-line Memory Module (DIMM) to store redundant check-bits and additional logic inside each DRAM controller to correct errors. Mission-critical systems commonly deploy strong ECC, called *Chipkill-correct*, to restore data from a completely failed chip [9]–[15]. As the error rate of individual DRAM chips has grown, DRAM vendors have introduced *on-die ECC* (O-ECC) to LPDDR4, DDR5, and HBM2E to maintain chip-level DRAM reliability [16]–[18]. O-ECC

The associate editor coordinating the review of this manuscript and approving it for publication was Christian Pilato.

utilizes extra cells within a DRAM chip to store redundancy and DRAM-internal circuits to correct errors. With *O-ECC*, a DRAM device can correct errors internally and maintain its external reliability level. Current DDR5 makes use of 6.25% extra cells to provide the weak protection of *Single Error Correction (SEC)* over a 128-bit data block [19], [20].

ECC alone, however, can be insufficient against the growing scale-related errors and increasing reliability requirements. While ECC can correct errors, it neither writes the corrected result back nor fixes the underlying fault. As non-transient faults repeatedly manifest errors, faults can accumulate over time and generate severe errors beyond the ECC capabilities. If bit faults accumulate within an *O-ECC* block, it will escape the single-bit correction capability. If faults accumulate across multiple chips, even the stronger Chipkill-correct *R-ECC* can fail to correct the error, yielding a system failure. Fault accumulations can be especially critical in systems with long lifetimes, such as self-driving vehicles.

Memory repair can improve reliability by retiring faulty regions and stopping fault accumulation. Traditionally, DRAM vendors have used spare rows/columns to repair a device from manufacturing faults and improve yield [21]. As the fault rate increases, however, industry and academia propose repair mechanisms against faults developed during operation. Recent DRAM standards [17], [22] support *Post Package Repair* to use spare rows against operational faults. Academic works have proposed remapping faulty DRAM cells using pinned last-level cache entries [23], [24], extra cells within a row [25]–[27], and an embedded cache inside a DRAM [28]. They show significant fault tolerance against increasing bit-level errors.

Prior repair mechanisms, however, have several drawbacks. First, some primarily target bit-level errors, yet field measurements show many faults manifest errors in ranges (e.g., a single-row fault). A bit-level remapping can quickly run out of its resources to remap cells individually. Second, others apply remapping at too-coarse granularity (e.g., a DRAM row or 64B cache line). With an increasing number of bit-level errors, they require excessive resources to retire single-bit faults. Third, with processing scaling, intermittent faults are increasing rapidly (as high as a bit error ratio of  $10^{-4}$  [29]). The prior work relies on static remapping and requires huge remapping overheads to retire all of the intermittent faults, although the faults manifest errors occasionally. Last but not least, all the previous works rely on the processor to detect errors and configure remapping, which leaks confidential error information outside the DRAM chip.

This paper proposes a novel DRAM micro-architecture called the *On-die Dynamic Remapping Cache (DRC)* to strongly and independently protect DRAM against intermittent faults. *DRC* embeds a small cache inside DRAM to remap the bad regions in a similar way to CiDRA [28], yet it provides stronger reliability without processor assistance. The superiority of *DRC* comes from two ideas: *dynamic remapping* and *fault type tracking*. *Dynamic remapping* is based on previous studies showing that most intermittent

faults are dormant most of the time, and the number of active faults at a time is relatively small [5], [30]–[32]. For example, an accelerated experiment on VRT shows the number of active faults over a 15-minute period is only 9 – 10% of total faults observed over a week [30]. In another study, the ratio of active faults is expected to be roughly down to 0.001% of total faults without accelerating [33]. Therefore, by tracking and remapping only active faults, *DRC* can substantially reduce the number of faults to be remapped without significant degradation in reliability.

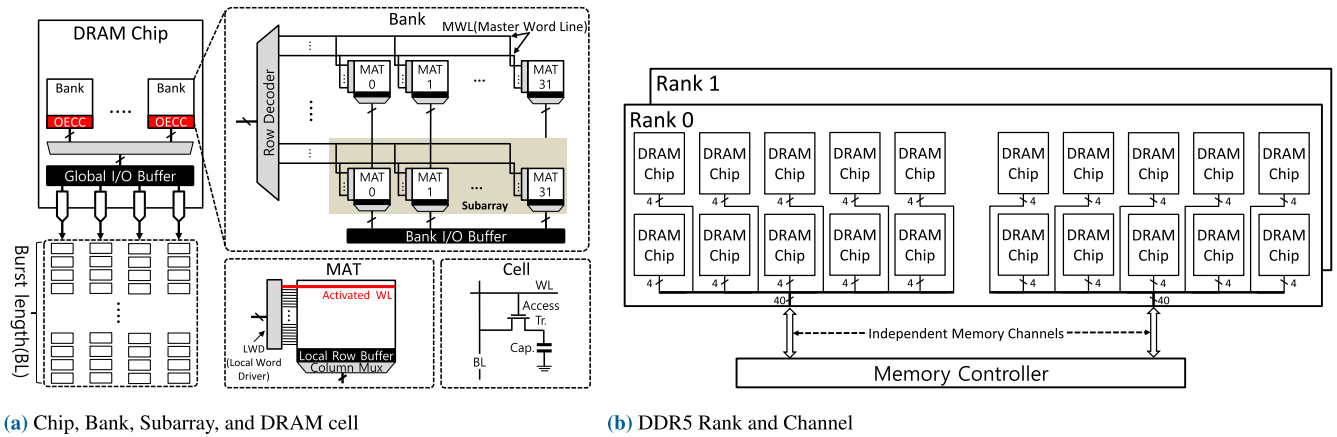
*Fault type tracking* tracks the severity of faults (i.e., whether it is a single-bit or multi-bit fault) to reduce the remapping overheads further. While single-bit error is correctable by *O-ECC*, multi-bit errors escape *O-ECC* and can damage system reliability. Therefore, utilizing remapping resources primarily for multi-bit faults can improve reliability with less overheads. *Fault type tracking* can identify multi-bit faults inside a DRAM at runtime. Its light-weight tracking mechanism incurs no performance overhead and requires no modification to the system.

*DRC* has several strengths over the state-of-the-art mechanisms. First, it provides strong and efficient protection. With dynamic remapping and fault type tracking, *DRC* minimizes fault accumulations and largely improves the reliability with small area overhead. Second, it is complete in that the reliability issue is isolated inside DRAM. *DRC* does not require any modification to the DRAM interface or any support from the host processor. It detects faulty regions and repairs faulty cells inside a DRAM chip. Moreover, DRAM vendors are reluctant to reveal their confidential error statistics, and *DRC* allows them to keep the information within the chip. Based on the strengths above, *DRC* can improve DRAM reliability beyond the state-of-the-art solutions.

The major contributions of this paper are as follows:

- We propose a novel repair mechanism, called *DRC*, to strongly and independently protect DRAM against growing intermittent faults. Intermittent faults are one of the dominant threats to memory reliability with process scaling. Detection and retiring of such faults are challenging with limited testing time and remapping resources. *DRC* provides efficient and strong protection against the growing faults.
- We propose a dynamic allocation scheme for remapping resources, called *dynamic remapping*. *DRC* deliberately keeps accessing remapped regions to track the activities of the underlying faults and utilizes the limited remapping resources to retire active faults.
- We propose a light-weight tracking mechanism to determine fault types online. As *O-ECC* reports errors one by one, *DRC* compares the syndrome of a new error against the old syndrome to identify multi-bit faults within a DRAM and priorities the faults in remapping.

The rest of this paper is organized as follows: Section II provides the background to facilitate the understanding of *DRC*. Section III analyzes some of the most related works



(a) Chip, Bank, Subarray, and DRAM cell

(b) DDR5 Rank and Channel

**FIGURE 1. An overview of DRAM hierarchy.**

in literature. Section IV provides an overview and details of *DRC* architecture. Section V evaluates *DRC*, demonstrating its ability to improve reliability beyond the state-of-the-art works. Section VI concludes the paper.

## II. BACKGROUND

This section reviews the concepts and terminology that are fundamental to *DRC*. Brief introductions to reliability, DRAM, DRAM faults, and ECC are presented below.

### A. TERMINOLOGY

A *fault* is a physical phenomenon or a defect that may cause an error, an *error* is a discrepancy between the intended and actual state of a system, and a *failure* is the event where a system deviates from its intended service [34]. A *transient* fault is introduced by some temporary environmental impetus (e.g., a high-energy particle strike). In contrast, a *permanent* fault is an irreversible physical defect that continually produces errors (e.g., a stuck-at-0 fault). An *intermittent* fault manifests errors at irregular intervals. Unlike permanent faults, intermittent faults occur only under specific conditions (e.g., elevated temperature) [35]. An intermittent fault is *active* when it produces an error and is *dormant* otherwise. An *inherent* fault is a non-transient fault developed during manufacturing, whereas an *operational* fault develops during operation.

Reliability indicates the continuity of service without a failure [34], often measured in *Failures In Time (FIT)*. One FIT is equal to one failure in a billion hours of operation. Reliability mechanisms can prevent an error from developing into a failure. ECC, for example, can allow a system to continue operation by correcting errors up to a certain degree. More severe errors, however, are either detected (*DUE, Detected Uncorrectable Error*) or left undetected (*SDC, Silent Data Corruption*).

### B. DRAM STRUCTURE

Figure 1a presents the hierarchical organization of a DRAM chip. A DRAM *cell* consists of one transistor and one

capacitor. A cell stores one bit of data by charging or discharging the capacitor. The charged capacitance leaks and eventually loses the data. The amount of time that the capacitor can retain its data is called the *retention time*. A DRAM controller periodically issues *refresh* commands for recharging the capacitors and preventing data loss.

A *bank* is a 2D array of cells sharing resources. A *row* is a group of cells sharing a *word-line*, while a *column* is a cell group sharing a *bit-line*. DRAM vendors add *spare rows* and *columns* to improve manufacturability. Faulty cells detected during manufacturing tests are remapped into one of the spares and repaired. After the tests, remapping information is programmed into non-volatile memory circuits inside the DRAM. The number of spare cells varies by vendors and products but is typically about 1% of the total cells [36], [37].

A DRAM *chip* can be broken into core and I/O parts. The DRAM *core* is responsible for storing data and utilizes multiple banks to overlap accesses and hide long access latency. The *I/O* part is responsible for transferring data to and from a processor. A DDR DRAM chip transfers data via 4 or 8 data pins (a  $\times 4$  or  $\times 8$  chip), and a *rank* is a collection of DRAM chips that are accessed together to provide the required data width. A *channel* is a group of ranks that share a data bus but where each rank can be controlled separately.

### C. DRAM FAULTS AND ERRORS

Traditionally, the DRAM process has been highly optimized to have few inherent faults. However, shrinking process technology has made DRAM more prone to faults and errors for several reasons. The smaller the cell transistor becomes with process shrinkage, the more vulnerable it is to minor defects. Smaller capacitors have more difficulty in maintaining the required charge for correct operation due to their limited aspect ratio, [5], [32] and their narrower pitches can result in disturbance errors, such as the *Row Hammer* [6] and *Write Disturbance Errors*. Increasing variability at smaller sizes also causes more severe fluctuations in retention time [33].

While the details of faults and errors are proprietary information, large-scale field studies provide some insights into

their distributions. Errors are relatively common, affecting 2% of servers each month and 9.6% of servers over twelve months [4]. Moreover, errors are severe, meaning that 0.03% of servers experience uncorrectable errors each month [4]. Another field measurement reports one DRAM error every 10 seconds, on average, in the 1.75 petaFLOP Jaguar supercomputer [1]. Large systems used in data centers and future supercomputers will likely suffer from more frequent DRAM errors as such systems aggressively increase memory capacity to scale with big-data workloads.

Faults can be classified based on the ranges they affect. A bit fault affects only a single cell. A row, column, and bank fault affect multiple cells in a row, column, or bank, respectively. Such faults can be caused by imperfect row/column decoders, sense-amplifiers, and I/O buffers [1], [29]. Prior field measurements provide detailed fault rates of these types in DDR2 and DDR3 [1], [2].

As process scaling continues, *intermittent faults* are of growing concern. Several field studies provide their characteristics [5], [7], [8], [30]: First, intermittent faults have been outgrowing other faults and will be the dominant challenge in memory reliability. Second, it is difficult to screen out all intermittent faults with testing. Although many intermittent faults can be detected by short-period testing, a considerable amount of them is still not detected after days of testing [30]. Third, most intermittent faults manifest as random bit errors [5], [7], [8], [30]. These small but frequent, intermittent faults make existing remapping schemes with a coarse granularity (e.g., row, column, cache line) inefficient. Last but not least, most intermittent faults have dormant and active states. An intermittent fault does not generate an error in the dormant state and generates errors occasionally in the active state. A couple of papers measured VRT and observed only a very small fraction of DRAM cells change their retention time at a given time period [7], [30].

#### D. ERROR CORRECTION CODE (ECC)

*Error Correction Codes (ECCs)* have long been used to protect data against errors. The most widely-used DRAM ECC is the *Single Error Correction – Double Error Detection (SEC-DED)* code. It can correct any 1-bit error and detect any 2-bit errors on each beat of a memory transfer. On a 64-bit data channel, 8b redundancy is needed for SEC-DED, leading to the industry standard of 72b ECC DIMM with 12.5% redundancy.

High-reliability systems use stronger Chipkill-correct protection. AMD chipkill-correct is one of the most efficient Chipkill-correct implementations; it can correct a  $\times 4$  chip error using a single 72b ECC DIMM. AMD chipkill-correct builds an 8-bit Reed-Solomon (RS) symbol out of two beats from the same  $\times 4$  chip, such that a chip error is confined to a single symbol per codeword [38]. There are other implementations of Chipkill-correct ECC that increase channel width (i.e., use two ECC DIMMs in lockstep) and detect all double-chip errors. Recent work has shown how RS code symbols

can be constructed differently to offer stronger overall protection than Chipkill-correct [39], [40].

Recently, DRAM vendors introduced *O-ECC* to LPDDR4, DDR5, and HBM2E [16]–[18]. These architectures add extra cells to the core and encoding/decoding circuitry to the bank or bank group [17], [19], [20]. Some implementations use 8-bit redundancy over 128-bit data to provide *Single Error Correction (SEC)* capability [19], [20]. They use a larger codeword (128-bit data) than DRAM access granularity (64-bit data) to amortize the redundancy down to 6.25%. However, the larger codeword size incurs power and performance penalties, such as overfetching and read-modify-writes. *O-ECC* does not provide Double Error Detection protection.

### III. PRIOR WORK

ECC alone can be insufficient against the growing errors, and memory repair can have a synergistic effect with ECC for tolerating persistent faults. DRAM vendors provide *Post Package Repair* in recent memories where a memory controller can change the spare row mapping during operation to remove operational faults [17], [22]. Platform developers (such as NVIDIA [42]) also use onboard non-volatile storage to remap faulty pages to achieve the same overall effect. While current industry practices tend toward coarse-grained remapping with rows and pages, Academia has proposed fine-grained remapping to retire increasing faults efficiently. This section provides some details of the work most relevant to *DRC*.

#### A. ECP, YODA, FLOWER/FaME

ECP [25] remaps permanent faults at a bit granularity using pointers to the faults. It adds pointer and spare cells to each row to store fault locations and remap the faulty cells. ECP, however, incurs a significant area overhead (11.9% extra cells to repair 6 bits per 512-bit row data) due to large pointers and row-dedicated remapping resources. Yoda [26] reduces the overhead of ECP by differentiating types of faults. FLOWER/FaME [27] improves the area efficiency of the bit-level sparing by replacing the pointers of ECP with a fault map. Its Bloom-filter-based fault map can share storage for fault locations across rows, such that it decreases the area overhead down to 4.3%. However, spare cells of FLOWER/FaME are still dedicated to a row, preventing more efficient resource utilization. Moreover, accessing the fault map incurs performance degradation, especially when data and the fault map are in the same bank.

#### B. ARCHSHIELD, CiDRA

Archshield [5] allocates a small region of DRAM to remap the faulty DRAM regions. However, the Archshield requires additional access to query fault map and incurs highly degraded performance. To overcome the performance degradation, CiDRA [28] embeds a small partitioned cache to remaps the faulty DRAM regions with cache lines. Because the CiDRA cache is inside DRAM chip, it can utilize a fine



**TABLE 1.** Feature Comparison of repair mechanisms.

|                                | Non-cache based remapping |                |                  | Cache based remapping |                |                 |                  |                             |
|--------------------------------|---------------------------|----------------|------------------|-----------------------|----------------|-----------------|------------------|-----------------------------|
|                                | ECP [25]                  | Archshield [5] | FLOWER/Fame [27] | CiDRA [28]            | FreeFault [23] | RelaxFault [24] | DEFCAM [41]      | DRC (Ours)                  |
| <b>Target Fault</b>            | DRAM                      | DRAM           | DRAM             | DRAM                  | DRAM           | DRAM            | SRAM             | <b>DRAM</b>                 |
| <b>Remapping Method</b>        | Static                    | Static         | Online           | Static                | Online         | Online          | Online & Dynamic | <b>Online &amp; Dynamic</b> |
| <b>Error Info.</b>             | Exposed                   | Exposed        | Exposed          | Exposed               | Exposed        | Exposed         | Exposed          | <b>Completely Hide</b>      |
| <b>Performance degradation</b> | 0%                        | upto 7%        | upto 2%          | 0%                    | upto 2%        | upto 2%         | unknown          | <b>0%</b>                   |

remapping granularity (64b). Also, CiDRA does not remap 1-bit faults in a chunk (typically a row) because they can be corrected by *O-ECC*. Instead, it remaps faults on a chunk with multiple error locations. CiDRA can tolerate a high fault rate with minimal area overhead with these strengths. However, CiDRA mainly targets increasing the yield, such that it is not tolerable against operational faults.

### C. FreeFault, RelaxFault

FreeFault [23] remaps a faulty DRAM region to a line in *Last-Level Cache (LLC)* during operation. After the remapping, it pins the cache line and does not access the faulty DRAM region. However, it is inefficient in resource utilization due to its coarse granularity (a 64B cache line). RelaxFault [24] also remaps a faulty DRAM region to cache lines in LLC. Unlike FreeFault, RelaxFault remaps only the faulty memory chip and keeps using healthy other devices in the rank. Coalescing multiple erroneous blocks into a single line reduces the number of pinned lines to 1/4 of FreeFault. Its coalescing can save remapping resources for ranged faults (e.g., a row fault), yet is ineffective against scattered faults (e.g., individual bit faults from process scaling).

### D. DEFCAM

DEFCAM [41] proposed a dynamic remapping mechanism for caches. It redirects access to a faulty cache set to one of the health sets. With more mapped blocks, the healthy remapping set may have more misses and degrade performance. With dynamic profiling of accesses, DEFCAM can change the healthy remapping set to a less-frequently accessed one to minimize performance degradation. However, it changes healthy remapping lines for performance, whereas DRC changes faulty remapped lines against intermittent faults.

The prior work has several drawbacks. First, they utilize static remapping to repair permanent faults, while intermittent faults increase rapidly. Future DRAMs will have bit error ratios as high as  $10^{-4}$  [29], and remapping all the faults requires an excessive amount of resources. Second, all the prior work relies on the processor to identify faults and/or remap faulty regions. DRAM error statistics are confidential information, and DRAM vendors do not want to share the information with their customers. However, prior work requires delivering raw error information to the processor and changing the DRAM interface.

Third, some prior work assumes perfect testing. CiDRA, for example, assumes all faulty cells are identified with testing and repaired before operation. Although many intermittent faults can be identified by short-period testing,

a considerable amount of them is still not detected after days of testing [30]. That necessitates an online repair mechanism during operation. Last but not least, field measurements show that DRAM faults have diverse patterns (e.g., single-bit, single-row, single-column), yet the remapping granularities are either too fine or too coarse. Some (ECP, YODA, and FLOWER/FaME) utilize bit-level remapping to efficiently retire individual bit faults yet quickly run out of their remapping resources with ranged faults (e.g., single-row fault). Others (CiDRA, FreeFault, and RelaxFault) utilize the cache-line granularity (64 bytes) and cannot retire individual bit faults efficiently. DRC utilizes a medium granularity (64 bits) to remap diverse patterns efficiently. The comprehensive features of repair mechanisms are summarized in Table 1.

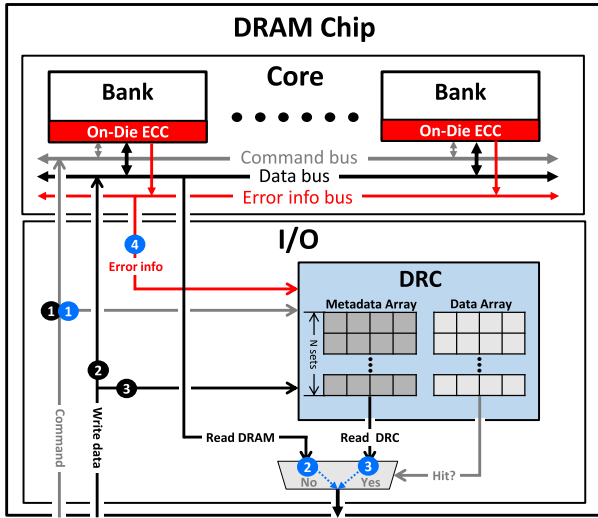
## IV. ON-DIE DYNAMIC REMAPPING CACHE

DRC is a novel DRAM micro-architecture that utilizes an on-die cache to repair memory from faults. With dynamic remapping and fault type tracking, DRC can efficiently remap a large volume of faults with small resources.

### A. OVERVIEW

Figure 2 presents an overall architecture with DRC. To remap faulty regions, DRC embeds a small cache on the DRAM global I/O, in a similar way to CiDRA. When a fault generates an error for the first time, *O-ECC* corrects the error, and DRC remaps the bad region into one of its cache lines. However, its efficiency comes from tracking the activity of faults. Following reads of the bad region will initiate accesses to *both* core and cache cells. The healthy cache cells provide data to be returned to a controller. The redundant access to the faulty core cells also provides data to *O-ECC*, which corrects an error, if any. DRC, however, discards the data after correction and uses only the correction information to check whether the region still suffers from the fault. This dual-path approach allows monitoring fault activity and severity without a performance penalty. Later, when the fault has been identified as dormant and a new active fault arrives, DRC reclaims the cache line and creates a new mapping for the new fault. By dynamically managing remappings, DRC can narrow down the remapping to active faults, which can be only a fraction of the faults while eliminating repeated errors from intermittent faults. An access to good core cells or cells under a dormant fault results in a DRC cache miss and serves the conventional core path with *O-ECC*. Section IV-B presents the details of data flow.

The cache in DRC is implemented using SRAM. We carefully selected SRAM over DRAM to allow inter-bank

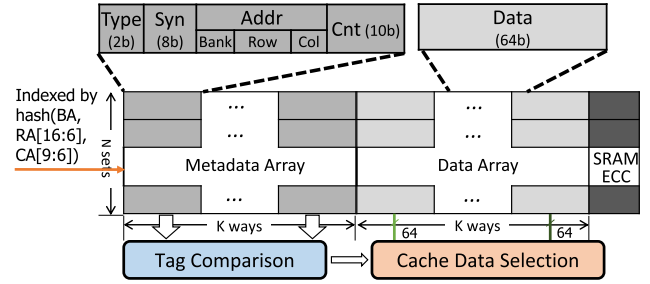


**FIGURE 2.** An overview of DRAM with DRC. DRC is located inside the DRAM global I/O. DRC and core process a read/write request in parallel. For a remapped region, DRC provides read data, yet correction information of core data allows DRC tracking fault activity.

sharing. While DRAM cells can provide a higher density, they require a similar bank organization to core cells to process overlapping requests in parallel. A cache line in a bank cannot remap a bad region in other banks. That can critically increase the size of required remapping resources, as fault distribution can be highly biased towards certain banks, and the number of banks in modern DRAM is 32 and increasing. Instead of dedicating resources to each bank and increasing the resource size to deal with the worst-case bank, SRAM-based DRC can reduce the overall overheads by sharing remapping resources across banks. SRAM also provides a shorter access latency, hiding DRC processing time in the DRAM access latency.

Implementing an SRAM using highly-optimized DRAM processes can be challenging. However, the small size of DRC cache allows less-dense implementations to meet highly-constrained DRAM design rules. As an example of SRAM implementation, an IP company embeds SRAM table within a DRAM to provide protection against RowHammer [43].

A DRC cache line consists of 64-bit data and 40-bit meta-data. The 64-bit data size matches the external access granularity (4-pin  $\times$  16-beats) and can serve a request by itself. The metadata includes *remapping type*, *syndrome*, *address* and a *counter*. The counter field is used for tracking fault activity (Section IV-D). The *remapping type* and *syndrome* fields are used to track underlying fault types (Section IV-E). As O-ECC reports errors one by one, DRC identifies severe multi-bit faults using the fields and prioritizes the faults. The metadata array and data array are accessed in parallel to reduce latency, and a separate ECC protects all metadata and data within a set to ensure correctness. The following subsections provide details of the DRAM access flow, cache structure, dynamic tracking, and fault type tracking.



**FIGURE 3.** DRC remapping boundaries and its cache structure. The target DRAM has 16-bit row address and 7-bit column address. BA/RA/CA represent bank/row/column addresses of an access/error, respectively.

### B. DATA FLOW

Figure 2 presents how a DRAM with DRC processes requests. For a write operation (1), DRAM duplicates the request and data to core and DRC. On the core path (2), O-ECC encodes the write data with redundancy and writes to the core cells. On the other path (3), DRC performs a cache lookup. With a match, it updates the corresponding data entry or ignores it otherwise.

On a read operation (1), the request is again blindly duplicated to the core and DRC. If it is a miss in DRC (2), this means the block has not been remapped, and the read data from the core becomes the final data (after O-ECC). If O-ECC reports an error on the core data (4), the corrected data is still used as the output, yet the dynamic remapping procedure starts remapping the region (Section IV-D). If it is a hit in DRC (3), it means the block has been remapped, and the data from the cache will be used as the final output. Later, when read data from core arrives and goes through O-ECC, (4), the read data is discarded. Yet, its error information updates the activity counter of the line in DRC. That allows for continuous monitoring of fault activity and tracking of fault type without an extra operation.

This parallel access also minimizes the latency penalty for DRC. Because a lookup time in the SRAM-based DRC is faster than reading from widely-distributed core cells, the read data from DRC arrives faster than core data and can hide the DRC processing time. The only latency increase we expect is a delay of the final 2:1 multiplexer, which selects the output data between the core and DRC based on the cache hit information. We assume that this does not affect the read latency in cycles in our evaluation (Section V).

### C. CACHE STRUCTURE

Figure 3 illustrates the cache organization of DRC and how it remaps the core cells. DRC utilizes a cache line size of 64 bits, which is the same size as the external access granularity. A cache line remaps a continuous (i.e., in the row direction) 64 bits, aligned to 64b boundaries. The meta-data of DRC consists of the following 4 fields.

### 1) TYPE

A 2-bit field to represent 3 types: *Invalid*, *Single*, and *Multi*. *Invalid* indicates that the line is unused. *Single* indicates that the line is remapping the faulty region, and the remapped region has reported only one error location. Based on subsequent error locations, a *Single* type can evolve into a *Multi*. *Multi* means that the block replaced the DRAM region, and its *O-ECC* block where the remapped region belongs has two or more error locations. Because the multi-bit faulty region is fatal to the reliability, a *Multi* block has a higher priority than *Single* and is less likely to get evicted from the cache.

### 2) SYNDROME

A 8-bit field to store previous error correction information from *O-ECC*. This field is used to compare the current error against the previous error and to evolve the Type field from *Single* to *Multi* (Section IV-E).

### 3) TAG

The address tag of the remapped block. This field contains all address bits, excluding the cache index for cache lookup.

### 4) COUNTER

A 10-bit saturating counter to indicate the activity of the underlying fault (Section IV-D). The higher the value, the more active the fault is. Based on this value, *DRC* selects a victim for replacement.

## D. DYNAMIC REMAPPING

The core of *DRC* is dynamic remapping, which minimizes remapping overheads by automatically changing remapped regions during operation. To track the activity of faults, *DRC* continues collecting error information of core cells and uses counters. This section provides the details of the mechanism.

Each read, whether the target is remapped or not, initiates access to core cells, and the returned data is checked by *O-ECC*. For an unmapped region (i.e., a cache miss), if *O-ECC* detects an error, it is the first error of the region (unless the region was evicted before). Then *DRC* allocates a cache line for the new fault and evicts an existing remapping that is most likely dormant if needed. It also updates its data array with the corrected data so that it can serve the following requests. For a remapped region (i.e., a cache hit), if *O-ECC* detects an error, it indicates that the fault is still active, and *DRC* increases the activity counter. If *O-ECC* cannot detect an error, the fault is likely to be dormant, and *DRC* decreases the counter. A non-error report on an unmapped region does not affect *DRC*.

For implementing this algorithm, we use the counter field in the metadata array. Storing the counter in the cache loses information when the line is evicted, but can significantly save storage overhead versus tracking activities of all regions, most of which are fault-free. The counters have four parameters: **Initial value**: An initial value for a new entry. **Faulty increment**: The amount by which the counter increases with an error report. **Healthy decrement**: The amount by

which the counter decreases with a non-error report. **Eviction threshold**: A threshold value for eviction. *DRC* selects a *Single* entry with the lowest counter in the target set as the victim for eviction. If all cache entries are *Multi*, *DRC* selects one whose value is lowest and less than the threshold. If all entries are *Multi* and greater than the threshold, the new fault is ignored.

Figure 4 presents a simple simulated example of how a counter changes its value with different fault types. For simplicity, we assume that a counter keeps tracking the same fault. In practice, the counter changes its monitoring region as regions are evicted. A transient fault starts with *Initial value* and keeps decreasing by *Healthy decrements*. A permanent fault, on the contrary, keeps increasing its value by *Faulty increments* and saturates at the maximum value. An intermittent fault that is mostly in active state generates errors frequently and fluctuates among high values. An infrequently-active fault can become easily dormant and fluctuates at a lower such that it can be easily evicted.

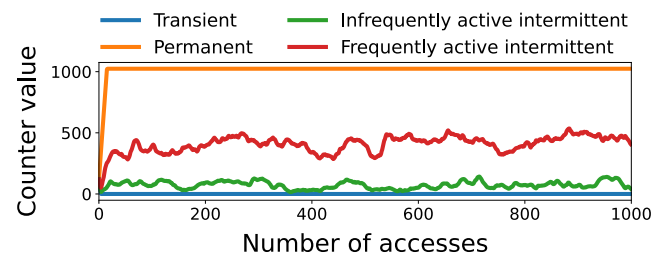


FIGURE 4. Examples of activity counter changes with different fault types.

## E. FAULT TYPE TRACKING

Although dynamic remapping can reduce remapping overheads by focusing on active faults, the number of active faults can still be high and require a considerable amount of remapping resources. To reduce remapping overheads further, *DRC* collaborates with *O-ECC*. Single-bit errors can be easily corrected by *O-ECC* until the underlying fault develops into a multi-bit one. *DRC* focuses on remapping multi-bit faults, which *O-ECC* cannot correct, and deprioritizes single-bit faults.

Identifying multi-bit faults can be simple with *R-ECC* or offline analyses. If a multi-bit fault manifests multi-bit errors, a Chipkill-correct *R-ECC* can immediately identify the fault with correction. If a multi-bit fault manifests single-bit errors on different positions over time, an offline analysis can easily identify the fault by drawing a 2D plot of error locations. However, inside a DRAM chip, identifying multi-bit faults is challenging. *O-ECC* can correct only one bit at a time, and provides weak detection on multi-bit errors. Also, tracking previous error locations within a DRAM incurs a large storage overhead to cost-sensitive DRAM vendors. The following subsections provide the details of how *DRC* detects multi-bit faults with small overheads.

## 1) MULTI-BIT FAULT DETECTION BY ON-DIE ECC

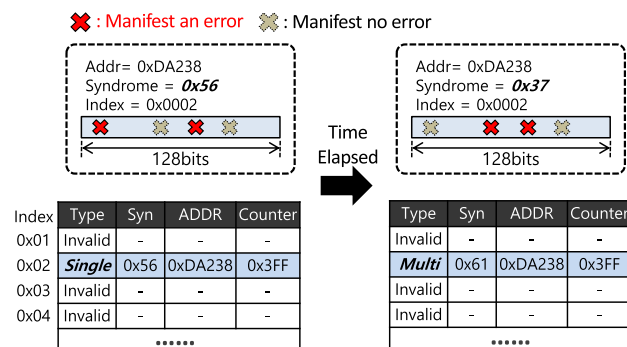
The SEC *O-ECC* provides limited-yet-useful detection capability against multi-bit errors. Although the ECC does not guarantee detection of all double bit errors, it can detect 46.7% of double and more severe errors.

It is because the ECC utilizes *shortened codes* to match the DRAM access granularity. The 8-bit redundancy in the ECC provides 255 non-zero syndromes to identify up to 255 errors. However, the ECC utilizes only 136 syndromes to correct single-bit errors on 128-bit data and 8-bit redundancy. If a multi-bit error generates a syndrome other than 136, *O-ECC* can identify the error as multi-bit. With simple circuit modifications on *O-ECC* to report unmatched syndromes, *DRC* can identify 46.7% of multi-bit faults.

## 2) MULTI-BIT FAULT DETECTION BY SYNDROME TRACKING

Detection coverage of *O-ECC* is still weak, and we augment the coverage with a novel tracking mechanism, called *Syndrome Tracking*. A multi-bit fault changes its error patterns over time with changes in data or cell properties. For example, writing 0 to a stuck-at 0 cell does not manifest an error, and an intermittent fault manifests errors occasionally. Therefore, syndromes from a multi-bit fault are not always identical. By storing the last error syndrome in the cache metadata, *DRC* can detect a change in the syndrome and identify multi-bit faults over time.

Figure 5 presents an example of *Syndrome Tracking*. At first, a multi-bit fault generated a multi-bit error. *O-ECC*, however, neither corrects nor detects the error but misinterprets it as a single error at another location (i.e., miscorrection). *DRC* receives the incorrect information and starts remapping the region with its type as *Single*. Later, new data is written, and the position of errors and the syndrome values change. On the following access to the remapped region, *DRC* compares its syndromes to that of the corresponding entry and recognizes its syndrome had been changed. Then, *DRC* confirms the remapping as having multi-bit faults and evolves the fault-type field from *Single* to *Multi*.



**FIGURE 5.** An example of detecting a multi-bit fault using syndrome tracking.

With *O-ECC* and *Syndrome Tracking*, *DRC* can efficiently identify multi-bit faults and pin their affected regions in the cache. There can be cases where a multi-bit fault gets its cache lines evicted before a type evolution due to many other single-bit errors. However, our evaluation shows that such cases are infrequent, even with small caches and limited associativity.

## F. PUTTING IT ALL TOGETHER

Figure 6 summarizes the overall process of *DRC*. In figure 6a, *O-ECC* reports a single-bit error for the first time. *DRC* then allocates an entry in the cache and marks it as *Single* and initializes its counter to *Initial value*. The allocation is 64b access granularity, half of the *O-ECC* codeword block. In figure 6b, another error on the faulty region is generated. Then the cache line evolves to *Multi* using multi-detection of *O-ECC* or *syndrome tracking*. In figure 6c illustrates the eviction procedure where a remapping with the lowest counter value is evicted. Note that *single*-type entry cannot evict the *Multi*-type entry, as multi-bit faults can escape *O-ECC* capability and significantly damage reliability.

## V. EVALUATION

This section estimates reliability, area, energy, and performance impacts of *DRC* and compares them against state-of-the-art schemes. Despite significant improvements in reliability, *DRC* has small hardware overheads and no performance degradation. Furthermore, it does so without any assistance from processors and requires no modification to current processors, modules, and interfaces.

The baseline configuration is a DDR5 channel with *O-ECC* and *R-ECC*. Each DDR5 chip has 8Gb capacity with 4 data pins and *O-ECC* to correct single-bit errors on 136b blocks (128b data + 8b redundancy). A DIMM has 80 data pins from 20 chips (i.e., the standard DDR5 ECC-DIMM with x4 chips), and a channel has two DIMMs to provide two ranks. *R-ECC* utilizes single-symbol correction on each 40-pin sub-channel to provide Chipkill-correct protection. The *R-ECC* builds an 8-bit Reed-Solomon symbol from two beats of data from the same chip (similar to AMD Chipkill-correct).

RelaxFault [24] represents the state-of-the-art CPU-cache-based repair mechanism. We allow up to 512KB LLC entries per DRAM channel to repair DRAM faults. CiDRA [28] represents the state-of-the-art repair mechanism based on a cache inside DRAM. Its cache configuration is not available in the paper, and we used 8KB capacity per chip, which is the same as *DRC* and roughly matches the per-chip capacity of RelaxFault. We assumed CiDRA performs a yearly scrubbing to detect faults and reclaim resources. FLOWER/FaME (F/F) [27] represents the state-of-the-art non-cache-based mechanism. It has 6 spare cells per 512-cell row and extra 4.3% area overhead to record fault locations.

## A. DEVICE-LEVEL RELIABILITY

The device-level reliability measures the estimated number of uncorrectable errors in a single chip. This metric is important



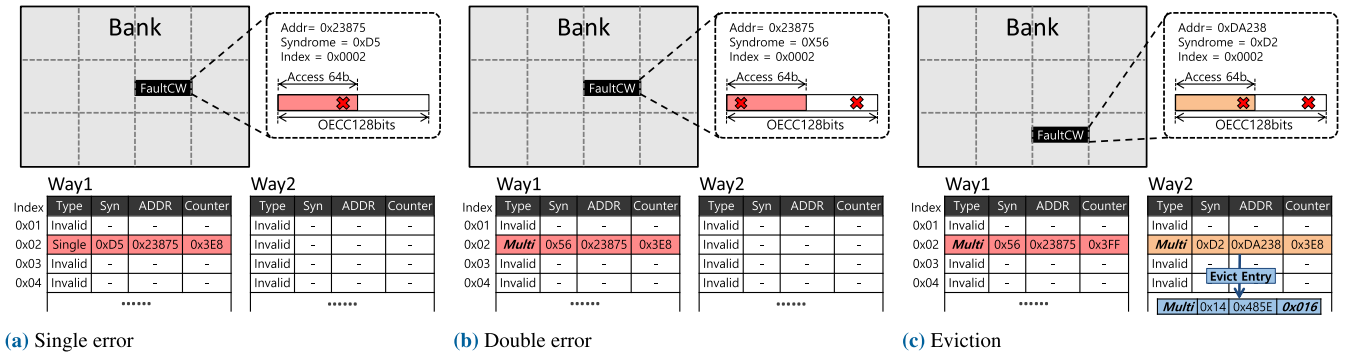


FIGURE 6. Summary of overall operation; dynamic remapping, fault type tracking and eviction.

to DRAM vendors, who want to hide errors inside a DRAM chip, and can indicate the level of pressure to *R-ECC*.

### 1) FAULT MODELING

In evaluating reliability, we apply two groups of faults; inherent and operational. Inherent faults develop during manufacturing (e.g., weak cells), and we model them as individual intermittent bit faults based on prior research [33]. We randomly inject the faults across a chip with a fixed BER. We use a BER of  $10^{-4}$  from an industry study [29] and assign an error manifestation probability to each fault based on a distribution from [7], [33], [44] (Figure 7b). We assume blocks with multiple bit faults are detected and retired during manufacturing tests. As a result, at time 0, a DRAM has all its blocks with up to 1 intermittent bit fault and is externally fault-free with *O-ECC*.

A new fault can arise during operation, such as a soft error from a cosmic ray strike and a hard error from transistor wear-out. We simplify the operational faults as intermittent single-bit ones (e.g., an unscreened weak cell) in this subsection but include other types of faults in the system-level reliability evaluation. The operational faults have different rates across processes and environments, and we test various rates. The rate starts from 12.5 FIT/Gb, which roughly matches the single-bit fault rate in a DDR3 error study, and increases up to 200 FIT/Gb. An operational fault may overlap with existing faults to generate a multi-bit error in a block.

### 2) SIMULATION FRAMEWORK

We modified the Monte Carlo DRAM fault-injection simulator used in [6] and [33]. This simulator is composed of three stages. The first stage injects new operational faults into a single DRAM chip with inherent faults, using a Poisson process. The second stage randomly generates errors on the affected blocks based on the existing faults and the history of accumulated new faults. The third stage receives the errors and processes *O-ECC* and repairs to find out whether the error is uncorrectable or not.

When a multi-bit error occurs inside a single chip and escapes *O-ECC*, the DRAM is no longer fault-free externally

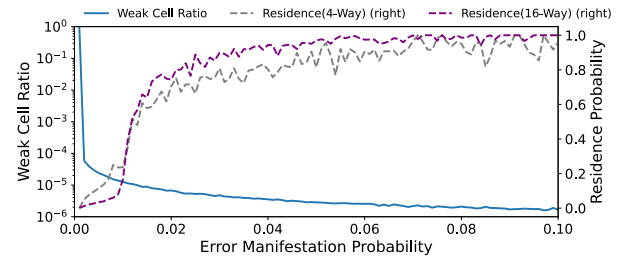
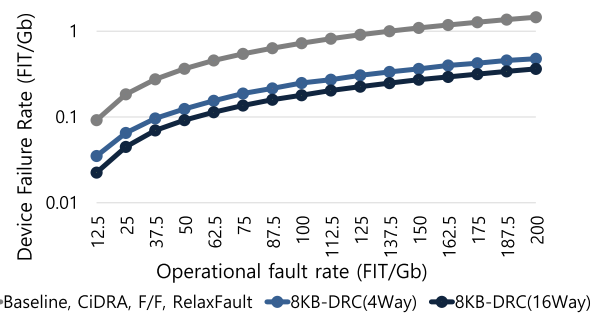


FIGURE 7. Device failure rate and its analysis.

and is regarded as a device failure. The failure occurs when the new and existing faults in the same block manifest errors simultaneously. Suppose only one manifests an error and the other is dormant. In that case, the single error is correctable by *O-ECC*, and *DRC* can remap the block before manifesting a multi-bit error with its syndrome tracking. The simulation evaluates the likelihood of observing an uncorrectable error in a single DRAM chip over five years and runs 1-billion times.

### 3) DEVICE-LEVEL RELIABILITY RESULT

Figure 7a represents the estimated device failure probability for different repair mechanisms. Overall, all the prior work provides the same device-level reliability as the baseline, while *DRC* improves reliability up to  $4\times$ . In the baseline (*O-ECC* without a repair), a new operational fault results in a failure if there is an existing fault in the block. With  $10^{-4}$

BER and 136b O-ECC blocks, the chances of a block having an existing bit fault are about 1%. That results in a failure rate of  $\sim 1$  FIT/Gb for a raw FIT rate of 100 FIT/Gb. All the prior work does not improve the reliability because they rely on the processor to identify and remap the fault. They require the error to be exposed to the processor first and cannot hide the confidential error information.

Despite the small capacity (8KB in an 8Gb DRAM), *DRC* can efficiently reduce the failure rate by 62% and 75% with 4-way and 16-way associativity, respectively. To analyze the efficiency, we measure the likelihood of cache residence across error manifestation probabilities (Figure 7b). The results show that mostly-active faults are highly likely to reside in the cache because of the dynamic remapping, whereas mostly-dormant ones are not. Prioritizing mostly-active faults over mostly-dormant ones can reduce the failure rate. A failure occurs when the new and existing faults in the same block manifest errors simultaneously. If the existing fault is dormant, the single error from the new fault is correctable by O-ECC, and *DRC* can remap the block before manifesting a multi-bit error using its syndrome tracking. Otherwise, a multi-bit error from the new and existing faults is uncorrectable to lead to a failure. *DRC* utilizes most of its small-capacity cache for active faults to prevent simultaneous multi-bit faults while relying on the O-ECC and syndrome tracking to remap the sequential multi-bit faults from mostly dormant ones.

## B. SYSTEM-LEVEL RELIABILITY

The system reliability is evaluated by estimating the number of failures in the system. We run fault-injection simulations with various fault types and apply O-ECC, repair, and R-ECC to find out whether the errors are correctable or not.

### 1) FAULT MODELING

In evaluating system-level reliability, we use the same inherent fault model from the previous section (Section V-A1), but operational faults are based on types and rates from a large-scale DDR3 measurement [3]. It provides detailed rates of various fault types (i.e., single-bit/-row/-column/-bank fault and multi-bank fault) and better represents real faults in the field. We increase the number of banks and rows to  $2\times$  and  $4\times$ , respectively, to match the 8Gb capacity (Table 2).

**TABLE 2.** DRAM fault types and rates (FIT/Gb) used in the evaluation (from DDR3 field study [3]).

| Fault Type    | Fault mode |           |
|---------------|------------|-----------|
|               | Transient  | Permanent |
| Single-bit    | 10.5       | 9.2       |
| Single-column | 0.0        | 1.5       |
| Single-row    | 0.4        | 1.8       |
| Single-bank   | 0.1        | 1.0       |
| Multiple-bank | 0          | 0.2       |
| Multiple-rank | 0.1        | 0.2       |

### 2) SIMULATION FRAMEWORK

We also modified the Monte Carlo DRAM fault-injection simulator in Section V-A2. The first stage injects operational faults into *multiple* DRAM chips with inherent faults based on the fault types and rates. The second stage randomly generates errors on each DRAM channel based on existing faults and the history of accumulated new faults. A non-transient fault accumulates and overlaps with future faults, while a transient fault does not accumulate and disappear after generating an error. It assumes each bit within a corrupted region has a 50% flipping probability (but the error-free pattern is excluded). The third stage receives the errors and processes O-ECC, repair, and R-ECC to find out whether the error is uncorrectable or not.

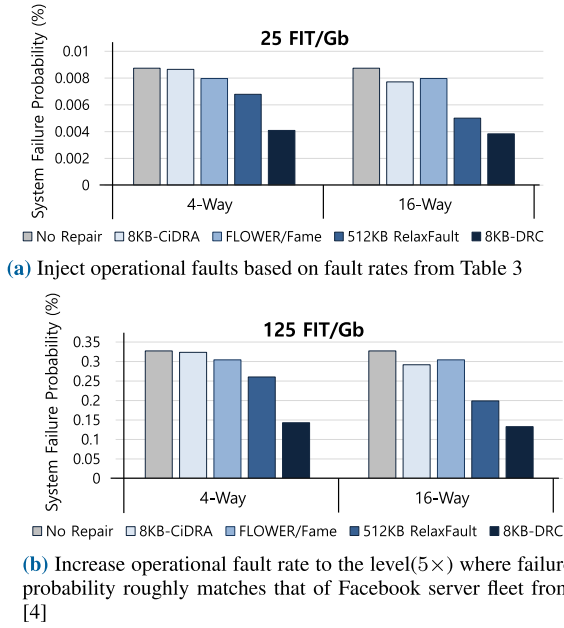
If a device generates a multi-bit error, the error is exposed outside the DRAM but is not regarded as a failure yet: it is regarded as a failure only if R-ECC cannot correct it. Suppose the error is corrected by the Chipkill-correct R-ECC and a repair mechanism remaps the underlying block before it manifests a multi-bit error. In that case, it does not result in a failure. Therefore, a failure occurs when multiple chips simultaneously manifest multi-bit errors on the same block. The simulation evaluates the likelihood of observing a failure in a system over 5 years and runs 1-billion times. We were not able to observe an SDC with 1 billion runs and provide uncorrectable failure rates only.

### 3) SYSTEM-LEVEL RELIABILITY RESULT

Figure 8a presents the system-level reliability of different repair mechanisms. The baseline (O-ECC + Chipkill-correct R-ECC + no repair) corrects single-bit and single-chip errors to reduce the system failure probability down to 0.0087% over 5 years. F/F (FLOWER/FaME) has large area overheads (3.13% for fault map and 1.2% for spare cells) but shows limited improvement in reducing failures against ranged faults; 0.0079% over 5 years. F/F can efficiently remap scattered bit faults with spare cells, yet its six spare cells per row quickly run out with a ranged fault (i.e., single-row/single-bank/multiple-bank). Without remapping, these faults can manifest errors beyond the single-error correction capability of O-ECC and can generate multi-chip errors with fault accumulation.

16-Way 8KB CiDRA also shows limited improvement in reducing failures; 0.0077% over 5 years. Although CiDRA can tolerate ranged faults with resource sharing, the offline-based remapping cannot stop fault accumulation immediately but have to wait for its yearly scrubbing. Among prior works, 4/16-way 512KB RelaxFault shows the biggest reduction of 0.00679%/0.005%, respectively, as it can prevent a new fault from the accumulation and remap ranged faults efficiently with its coalescing unit.

Despite the smaller total capacity (320KB) than 512KB RelaxFault, 4/16-way *DRC* reduces failure probability further down to respectively 0.004/0.0038%. The superiority comes from improving device-level reliability with its



**FIGURE 8. System reliability comparison for memory repair mechanisms. This bar graph shows the likelihood of observing system failure per node over 5 years.**

dynamic remapping and syndrome tracking. Additionally, the medium granularity (64b) of *DRC* is efficient in remapping bit-level faults and ranged faults, while RelaxFault remaps at a cacheline granularity (64B).

We also run experiments with  $5\times$  operational fault rates for future memory scaling. Figure 8b presents the system reliability with the accelerated fault rates. The baseline shows a failure probability of 0.32% where the failure probability roughly matches with Facebook server fleets [4]. The overall reliability trend equals to the previous figure 8a, except the reliability gap between RelaxFault and *DRC* gets widened. While *DRC* reduces 24% failure of RelaxFault in 25 FIT/Gb, *DRC* reduces more failure in 125 FIT/Gb; 34% of RelaxFault, meaning that *DRC* is more effective to reliable future memory system.

### C. CACHE SET-ASSOCIATIVITY

The number of ways of *DRC* can affect reliability by prematurely evicting a faulty block between its error manifestations. We estimate the likelihood of *DRC* failures against random bit errors with different set-associativities to analyze the impact.

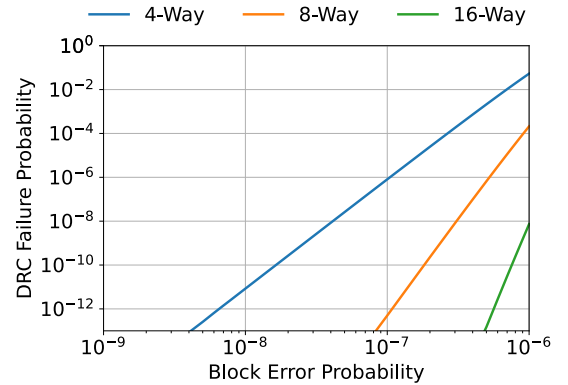
Let *DRC* has a *R*-block capacity using *W*-way associativity and  $S(= \frac{R}{W})$  sets. A DRAM with a capacity of *C* blocks evenly distributes its blocks to the cache sets so that a set has  $M(= \frac{C}{S})$  mapped blocks. The likelihood of having more-than-*W* erroneous blocks within a set can be estimated as

$$P_{fail\_set} = \sum_{i=W+1}^M M C_i \cdot P_{error\_block}^i \cdot (1 - P_{error\_block})^{M-i}, \quad (1)$$

where  $P_{error\_block}$  represents the likelihood of a block having at least one error and *i* represents the number of erroneous blocks. The probability of *DRC* having no failing set over *S* sets can be estimated as follows.

$$P_{fail\_DRC} = 1 - (1 - P_{fail\_set})^S \quad (2)$$

Figure 9 presents  $P_{fail\_DRC}$  with different set-associativities for an 8Gb DRAM with 8KB *DRC*. Increasing set associativity (decreasing the number of sets) for the same capacity can improve reliability by orders of magnitude. At  $10^{-6}$   $P_{error\_block}$ , for instance, 4-way (256 sets) and 16-way (64 sets) associativities have  $P_{fail\_DRC}$  as  $5 \times 10^{-2}$  and  $7 \times 10^{-9}$ , respectively. Not only the higher associativity remaps more random bit errors within a set, but also the reduced number of sets decreases the *DRC* failure probability. However, the impact of associativity reduces as some errors are more frequent than others (Section V-A3) and multi-bit faults generate multi-bit errors instantaneously (Section V-B3). Increasing the set-associativity consumes more energy, yet the following evaluation shows *DRC* consumes a small fraction of DRAM power due to its small capacity.



**FIGURE 9. A comparison of *DRC* failure probabilities against random bit errors with different cache set-associativities.**

### D. AREA AND ENERGY

We estimate area and energy overheads of *DRC* using CACTI 7.0 [45] on 22nm process. The baseline is an 8Gb DRAM with *O-ECC* and row/column spares. The spares use 1% of total cells and occupy 0.5% of the chip area. Figure 10 shows the overhead of the embedded cache. 8KB *DRC* occupies  $0.012\text{mm}^2$ , which translates to 0.012% of a  $105.3\text{mm}^2$  8Gb DRAM chip. Although it is a small number, the overhead could be improved to negative (−0.34%) by replacing row/column spares with *DRC*. The repair capability of *DRC* is not limited to operational faults and can be used to repair inherent faults detected during manufacturing. Using some non-volatile circuitry to pin defective faults after testing can eliminate most, if not all, spares. After replacing all the spares, the overall chip area can decrease up to 99.6% than the baseline. In terms of energy, 4/16-Way 8KB *DRC* consumes 0.0031/0.0038nJ additional energy per read access, which is 0.57%/0.70% of the DRAM read energy, respectively.

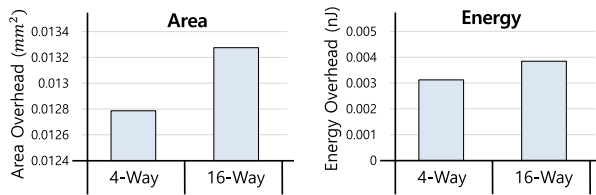


FIGURE 10. Estimations of DRC area and energy overheads.

TABLE 3. A comparison of Reliability (R), Area (A), Performance (P), and two RAP metrics (M1-M2).

| Repair Mechanism          | R    | A    | P    | M1   | M2   |
|---------------------------|------|------|------|------|------|
| No Repair                 | 0.43 | 0.99 | 1.00 | 0.43 | 0.19 |
| CiDRA (8KB 16-Way)        | 0.48 | 1.00 | 1.00 | 0.45 | 0.21 |
| FLOWER/Fame               | 0.50 | 1.04 | 0.98 | 0.49 | 0.24 |
| RelaxFault (512KB 16-Way) | 0.76 | 1.00 | 0.98 | 0.74 | 0.57 |
| DRC (8KB 16-Way)          | 1    | 1    | 1    | 1    | 1    |

### E. PERFORMANCE

To estimate the performance impact of *DRC*, we measure *DRC* access time using CACTI 7.0 [45]. CACTI estimates the access time of *DRC* cache is less than 0.2ns, which is only a small fraction of access time to core cells. Because of the parallel accesses to the core and *DRC*, *DRC* processing time can be hidden and is not on the critical path.

The only component added to the critical path is the 2:1 mux to choose the final output. Due to the lack of a publicly available DRAM library, we use UMC 28nm RVT technology to estimate the delay. A 2:1 mux with  $16\times$  drive strength has a cell delay of 22ns. Based on the information, we expect the additional delay to be 30ps after process scaling and customization and does not affect read latency in cycles (420ps clock period in DDR5-4800) and system performance. Compared to *DRC* without performance degradation, some of the prior work slows down the execution time. At  $10^{-4}$  BER, for example, FLOWER/Fame and RelaxFault report up to 2% slow-downs due to fault-map lookup and reduced LLC capacity, respectively.

### F. RELIABILITY, AREA AND PERFORMANCE

To quantify the overall benefits of *DRC*, we use metrics derived from [41]. The derived metric combines three primary metrics, reliability (R), area (A), and performance (P), into two combined metrics (M1, M2). The R metric is the likelihood of 5-year system survival probability (Figure 8a), the A metric is the estimated chip areas increase, and the P metric is the worst execution time reported at  $10^{-4}$  BER. All the metrics are normalized to *DRC*. Then we combine these terms into M1 ( $=R \cdot A^{-1} \cdot P$ ) and M2 ( $=R^2 \cdot A^{-1} \cdot P$ ). The  $A^{-1} \cdot P$  represents the area efficiency, and we multiply the efficiency with R and  $R^2$  to weigh reliability differently (e.g., M2 for reliability-critical systems). Table 3 compares the overall effectiveness. *DRC* shows the best (the highest) results in both M1 and M2 with the highest reliability, a negligible chip area increase, and no performance degradation.

### VI. CONCLUSION

This paper proposes and analyzes a novel DRAM micro-architecture, called *DRC* (On-die Dynamic Remapping Cache) to improve memory reliability against growing *intermittent faults*. Intermittent faults have become the dominant threat to memory reliability. To achieve high reliability against their repeated errors, retiring regions under intermittent faults is desirable but can cost too many overheads. Based on *Dynamic Remapping* and *Fault Type Tracking*, *DRC* reduces the overheads by decreasing the number of faults to be remapped and the number of resources for remapping, respectively. With the improved efficiency, *DRC* aggressively retires active faults and can achieve highly improved reliability without significant overheads in area and power and does not hurt performance. It does not require a change in processor, module, or interface. It also allows DRAM vendors to keep confidential error information inside their products.

### ACKNOWLEDGMENT

The EDA tool was supported by the IC Design Education Center (IDEC), South Korea.

### REFERENCES

- [1] V. Sridharan and D. Liberty, "A study of DRAM failures in the field," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2012, pp. 1–11.
- [2] V. Sridharan, J. Stearley, N. De Bardeleben, S. Blanchard, and S. Gurumurthi, "Feng shui of supercomputer memory positional effects in dram and SRAM faults," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2013, pp. 1–11.
- [3] V. Sridharan, N. De Bardeleben, S. Blanchard, K. B. Ferreira, J. Stearley, J. Shalf, and S. Gurumurthi, "Memory errors in modern systems: The good, the bad, and the Ugly," *ACM Sigplan Notices*, vol. 50, no. 4, pp. 297–310, Mar. 2015.
- [4] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "Revisiting memory errors in large-scale production data centers: Analysis and modeling of new trends from the field," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2015, pp. 415–426.
- [5] P. J. Nair, D.-H. Kim, and M. K. Qureshi, "ArchShield: Architectural framework for assisting DRAM scaling by tolerating high error rates," in *Proc. 40th Annu. Int. Symp. Comput. Archit.*, Jun. 2013, pp. 72–83.
- [6] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 3, pp. 361–372, 2014.
- [7] S. Khan, D. Lee, Y. Kim, A. R. Alameldeen, C. Wilkerson, and O. Mutlu, "The efficacy of error mitigation techniques for dram retention failures: A comparative experimental study," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 42, no. 1, pp. 519–532, 2014.
- [8] A. Bacchini, M. Rovatti, G. Furano, and M. Ottavi, "Characterization of data retention faults in DRAM devices," in *Proc. IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst. (DFT)*, Oct. 2014, pp. 9–14.
- [9] *HP Advanced Memory Error Detection Technology*, HP Enterprise, Spring, TX, USA, 2011.
- [10] *Chipkill Memory*, Int. Bus. Mach., Armonk, NY, USA, 2012.
- [11] *Bios and Kernel Developer's Guide (BKDG) for AMD Family 15h Models 00h-0fh Processors*, Advanced Micro Devices, Santa Clara, CA, USA, 2013.
- [12] *Intel Xeon Processor E7 Family: Reliability, Availability, and Serviceability*, Intel Corp., Santa Clara, CA, USA, 2011.
- [13] *How Memory RAS Technologies Can Enhance the Uptime of HPE Proliant Servers*, Hewlett Packard Enterprise, Spring, TX, USA, 2016.
- [14] *Intel Xeon Processor, Sun Microsystems*, Santa Clara, CA, USA, 2007. [Online]. Available: <http://www.oracle.com/technetwork/systems/opensparc/t2-06-opensparc2-core-microarch-1537749.html>



- [15] Oracle's SPARC S7 Server Architecture, Oracle Corp., Austin, TX, USA, 2016. [Online]. Available: <https://www.oracle.com/a/ocom/docs/servers/sparc/s7-architecture.pdf>
- [16] Low Power Double Data Rate 4 (LPDDR4), Standard JESD209-4C, Joint Electronic Device Engineering Council, 2020.
- [17] DDR5 SDRAM Standard, Standard JESD79-5, Joint Electronic Device Engineering Council, 2020.
- [18] C.-S. Oh *et al.*, "22.1 A 1.1 V 16 GB 640 GB/s HBM2E DRAM with a data-bus window-extension technique and a synergetic on-die ECC scheme," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2020, pp. 330–332.
- [19] K. C. Chun, Y.-G. Chu, J.-S. Heo, T.-S. Kim, S. Kim, H.-K. Yang, M.-J. Kim, C.-K. Lee, J. Kim, and H. Yoon, "A 16 GB LPDDR4X SDRAM with an NBTI-tolerant circuit solution, an SWD PMOS GIDL reduction technique, an adaptive gear-down scheme and a metastable-free DQS aligner in a 10 nm class dram process," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2018, pp. 206–208.
- [20] T. Oh, "A 3.2 Gbps/pin 8 Gbit 1.0 V LPDDR4 SDRAM with integrated ECC engine for sub-1 V DRAM core operation," *IEEE J. Solid-State Circuits*, vol. 50, no. 1, pp. 178–190, Jan. 2015.
- [21] T. Takahashi, T. Sekiguchi, R. Takemura, S. Narui, H. Fujisawa, S. Miyatake, M. Morino, K. Araim, S. Yamada, S. Shukuri, M. Nakamura, Y. Tadaki, K. Kajigaya, K. Kimura, and K. Itoh, "A multigigabit dram technology with 6F/Sup 2/open-bitline cell, distributed overdriven sensing, and stacked-flash fuse," *IEEE J. Solid-State Circuits*, vol. 36, no. 11, pp. 1721–1727, Nov. 2001.
- [22] DDR4 SDRAM Standard, Standard JESD79-4C, Joint Electronic Device Engineering Council, 2020.
- [23] D. W. Kim and M. Erez, "Balancing reliability, cost, and performance tradeoffs with FreeFault," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2015, pp. 439–450.
- [24] D. W. Kim and M. Erez, "RelaxFault memory repair," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 645–657.
- [25] S. Schechter, G. H. Loh, K. Straus, and D. Burger, "Use ECP, not ECC, for hard failures in resistive memories," *ACM SIGARCH Comput. Archit.*, vol. 38, no. 3, pp. 141–152, Jun. 2010.
- [26] J. Zhang, D. Kline, L. Fang, R. Melhem, and A. K. Jones, "Yoda: Judge me by my size, do you?" in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, Nov. 2017, pp. 395–398.
- [27] D. Kline, J. Zhang, R. Melhem, and A. K. Jones, "FLOWER and FaME: A low overhead bit-level fault-map and fault-tolerance approach for deeply scaled memories," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 356–368.
- [28] Y. H. Son, S. Lee, O. Seongil, S. Kwon, N. S. Kim, and J. H. Ahn, "CiDRA: A cache-inspired DRAM resilience architecture," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2015, pp. 502–513.
- [29] S. Cha, O. Seongil, H. Shin, S. Hwang, K. Park, S. J. Jang, J. S. Choi, G. Y. Jin, Y. H. Son, H. Cho, J. H. Ahn, and N. S. Kim, "Defect analysis and cost-effective resilience architecture for future DRAM devices," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2017, pp. 61–72.
- [30] M. K. Qureshi, D.-H. Kim, S. Khan, P. J. Nair, and O. Mutlu, "AVATAR: A Variable-Retention-Time (VRT) aware refresh for DRAM systems," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2015, pp. 427–437.
- [31] S.-K. Park, "Technology scaling challenge and future prospects of DRAM and NAND flash memory," in *Proc. IEEE Int. Memory Workshop (IMW)*, May 2015, pp. 1–4.
- [32] U. Kang, H.-S. Yu, C. Park, H. Zheng, J. Halbert, K. Bains, S. Jang, and J. S. Choi, "Co-architecting controllers and dram to enhance dram process scaling," *Memory Forum*, vol. 14, pp. 1–14, Jun. 2014.
- [33] S.-L. Gong, J. Kim, and M. Erez, "DRAM scaling error evaluation model using various retention time," in *Proc. 47th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshops (DSN-W)*, Jun. 2017, pp. 177–183.
- [34] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Trans. Depend. Sec. Comput.*, vol. 1, no. 1, pp. 11–33, Jan./Mar. 2004.
- [35] C. Constantinescu, "Impact of deep submicron technology on dependability of VLSI circuits," in *Proc. Int. Conf. Dependable Syst. Netw.*, Jun. 2002, pp. 205–209.
- [36] K. Cho, W. Kang, H. Cho, C. Lee, and S. Kang, "A survey of repair analysis algorithms for memories," *ACM Comput. Surv.*, vol. 49, no. 3, pp. 1–41, Oct. 2016.
- [37] W. Kang, H. Cho, J. Lee, and S. Kang, "A BIRA for memories with an optimal repair rate using spare memories for area reduction," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 11, pp. 2336–2349, Nov. 2014.
- [38] *Bios and Kernel Developers Guide (BKDG) for AMD Family 15h Models 00h-0fh Processors*, AMD Inc., Santa Clara, CA, USA, 2013.
- [39] J. Kim, M. Sullivan, and M. Erez, "Bamboo ECC: Strong, safe, and flexible codes for reliable computer memory," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2015, pp. 101–112.
- [40] J. Kim, M. Sullivan, S. Lym, and M. Erez, "All-inclusive ECC: Thorough end-to-end protection for reliable computer memory," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 622–633.
- [41] H. Lee, S. Cho, and B. R. Childers, "DEFCAM: A design and evaluation framework for defect-tolerant cache memories," *ACM Trans. Archit. Code Optim.*, vol. 8, no. 3, pp. 1–29, Oct. 2011.
- [42] D. Tiwari, S. Gupta, G. Gallarno, J. Rogers, and D. Maxwell, "Reliability lessons learned from GPU experience with the Titan supercomputer at Oak ridge leadership computing facility," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2015, pp. 1–12.
- [43] T. R. Halfhill, "Upmem nails RowHammer: French company offers defense against DRAM vulnerability," Microprocessor Rep., Linley Group, Mountain View, CA, USA, Jun. 2021. [Online]. Available: <https://www.upmem.com/microprocessor-report-linley-on-upmems-row-hammer-solution/>
- [44] K. Kim and J. Lee, "A new investigation of data retention time in truly nanoscaled DRAMs," *IEEE Electron Device Lett.*, vol. 30, no. 8, pp. 846–848, Aug. 2009.
- [45] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, pp. 1–25, Jul. 2017.



**SANGJAE PARK** received the B.S. degree in electronic and electrical engineering from Sungkyunkwan University, South Korea, in 2021, where he is currently pursuing the M.S. degree in electrical and computer engineering. His current research interest includes memory reliability.



**JUNGRAE KIM** (Senior Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Seoul National University, South Korea, and the Ph.D. degree from the University of Texas at Austin, USA. From 2003 to 2016, he was a Senior Engineer with Samsung Electronics, where he developed application processors. From 2016 to 2020, he was a Senior Software Engineer with Microsoft Corporation, USA, architecting data center accelerators. He joined Sungkyunkwan University as an Assistant Professor, in 2020. His current research interests include heterogeneous architecture, memory subsystems, and reliability.

...